Backup slides



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



mmap



mmap

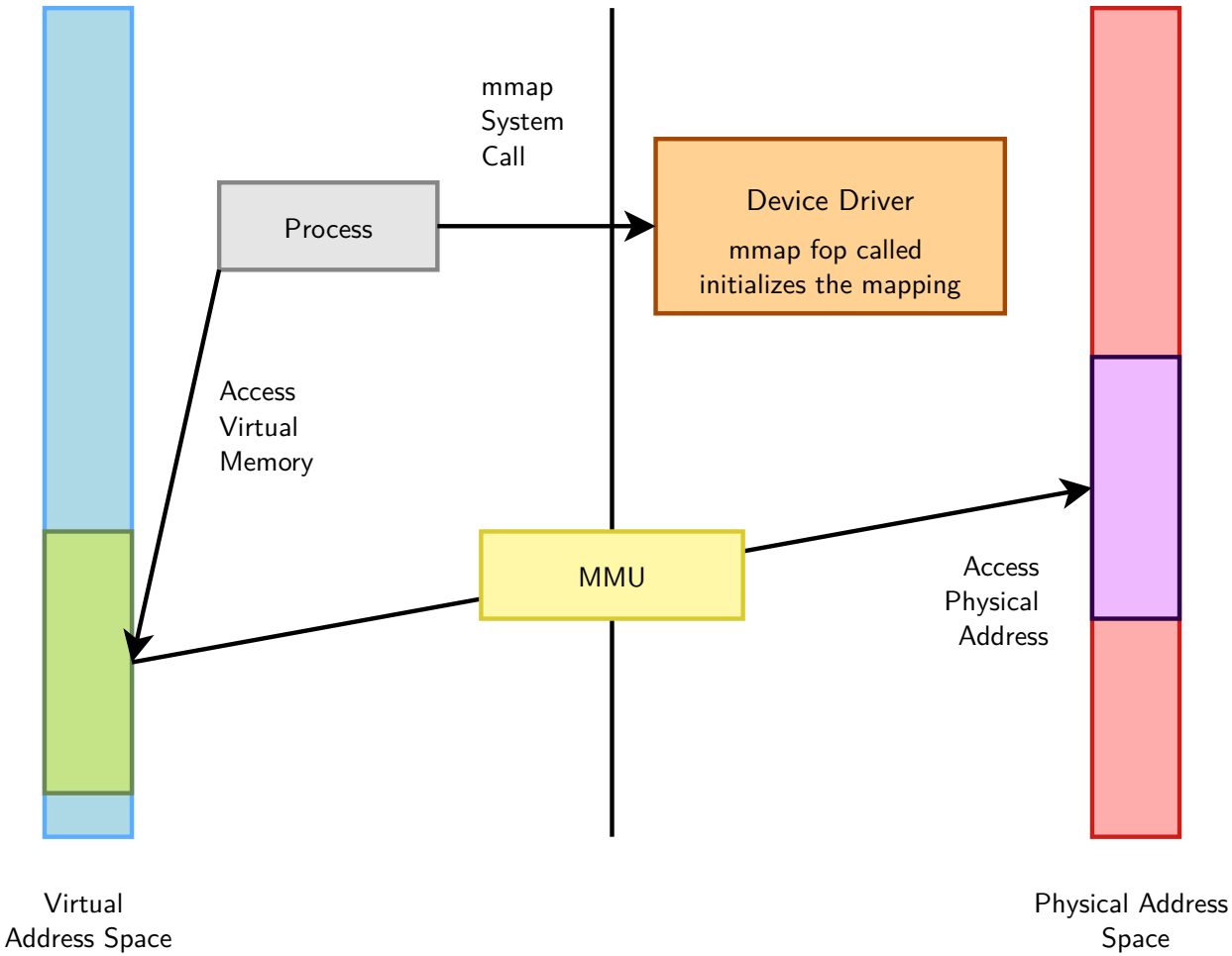
- Possibility to have parts of the virtual address space of a program mapped to the contents of a file
- Particularly useful when the file is a device file
- Allows to access device I/O memory and ports without having to go through (expensive) read, write or ioctl calls
- One can access to current mapped files by two means:
  - `/proc/<pid>/maps`
  - `pmap <pid>`



/proc/<pid>/maps

```
       start-end          perm offset major:minor inode  mapped file name
...
7f4516d04000-7f4516d06000 rw-s 1152a2000 00:05 8406   /dev/dri/card0
7f4516d07000-7f4516d0b000 rw-s 120f9e000 00:05 8406   /dev/dri/card0
...
7f4518728000-7f451874f000 r-xp 00000000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f451874f000-7f451894f000 ---p 00027000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f451894f000-7f4518951000 r--p 00027000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f4518951000-7f4518952000 rw-p 00029000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
...
7f451da4f000-7f451dc3f000 r-xp 00000000 08:01 1549    /usr/bin/Xorg
7f451de3e000-7f451de41000 r--p 001ef000 08:01 1549    /usr/bin/Xorg
7f451de41000-7f451de4c000 rw-p 001f2000 08:01 1549    /usr/bin/Xorg
...
```

mmap overview





How to Implement mmap - User space

- Open the device file
- Call the `mmap` system call (see `man mmap` for details):

```
void * mmap(
    void *start,   /* Often 0, preferred starting address */
    size_t length, /* Length of the mapped area */
    int prot,      /* Permissions: read, write, execute */
    int flags,     /* Options: shared mapping, private copy... */
    int fd,        /* Open file descriptor */
    off_t offset   /* Offset in the file */
);
```

- You get a virtual address you can write to or read from.



How to Implement mmap - Kernel space

- Character driver: implement an `mmap` file operation and add it to the driver file operations:

```
int (*mmap) (
    struct file *,           /* Open file structure */
    struct vm_area_struct *  /* Kernel VMA structure */
);
```

- Initialize the mapping.
  - Can be done in most cases with the remap_pfn_range() function, which takes care of most of the job.



remap_pfn_range()

- *pfn*: page frame number
- The most significant bits of the page address (without the bits corresponding to the page size).

```
#include <linux/mm.h>

int remap_pfn_range(
    struct vm_area_struct *, /* VMA struct */
    unsigned long virt_addr, /* Starting user
                              * virtual address */
    unsigned long pfn,       /* pfn of the starting
                              * physical address */
    unsigned long size,      /* Mapping size */
    pgprot_t prot            /* Page permissions */
);
```



Simple mmap implementation

```
static int acme_mmap
    (struct file * file, struct vm_area_struct *vma)
{
      size = vma->vm_end - vma->vm_start;

      if (size > ACME_SIZE)
          return -EINVAL;

      if (remap_pfn_range(vma,
                  vma->vm_start,
                  ACME_PHYS >> PAGE_SHIFT,
                  size,
                  vma->vm_page_prot))
          return -EAGAIN;

      return 0;
    }
```



devmem2

- https://bootlin.com/pub/mirror/devmem2.c, by Jan-Derk Bakker
- Very useful tool to directly peek (read) or poke (write) I/O addresses mapped in physical address space from a shell command line!
  - Very useful for early interaction experiments with a device, without having to code and compile a driver.
  - Uses `mmap` to `/dev/mem`.
  - Examples (`b`: byte, `h`: half, `w`: word)
    - `devmem2 0x000c0004 h` (reading)
    - `devmem2 0x000c0008 w 0xffffffff` (writing)
  - `devmem` is now available in BusyBox, making it even easier to use.



mmap summary

- The device driver is loaded. It defines an `mmap` file operation.
- A user space process calls the `mmap` system call.
- The `mmap` file operation is called.
- It initializes the mapping using the device physical address.
- The process gets a starting address to read from and write to (depending on permissions).
- The MMU automatically takes care of converting the process virtual addresses into physical ones.
- Direct access to the hardware without any expensive `read` or `write` system calls



Useful general-purpose kernel APIs



Memory/string utilities

- In include/linux/string.h
  - Memory-related: memset(), memcpy(), memmove(), memscan(), memcmp(), memchr()
  - String-related: strcpy(), strcat(), strcmp(), strchr(), strrchr(), strlen() and variants
  - Allocate and copy a string: kstrdup(), kstrndup()
  - Allocate and copy a memory area: kmemdup()
- In include/linux/kernel.h
  - String to int conversion: simple_strtoul(), simple_strtol(), simple_strtoull(), simple_strtoll()
  - Other string functions: sprintf(), sscanf()



Linked lists

- Convenient linked-list facility in include/linux/list.h
  - Used in thousands of places in the kernel
- Add a struct list_head member to the structure whose instances will be part of the linked list. It is usually named `node` when each instance needs to only be part of a single list.
- Define the list with the LIST_HEAD() macro for a global list, or define a struct list_head element and initialize it with INIT_LIST_HEAD() for lists embedded in a structure.
- Then use the `list_*()` API to manipulate the list
  - Add elements: list_add(), list_add_tail()
  - Remove, move or replace elements: list_del(), list_move(), list_move_tail(), list_replace()
  - Test the list: list_empty()
  - Iterate over the list: `list_for_each_*()` family of macros



Linked lists examples 1/2

drivers/i2c/busses/i2c-stm32f7.c

```
/**
 * struct stm32f7_i2c_timings - private I2C output parameters
 * @node: List entry
 * @presc: Prescaler value
 * @scldel: Data setup time
 * @sdadel: Data hold time
 * @sclh: SCL high period (master mode)
 * @scll: SCL low period (master mode)
 */
struct stm32f7_i2c_timings {
  struct list_head node;
  u8 presc;
  u8 scldel;
  u8 sdadel;
  u8 sclh;
  u8 scll;
};
```



Linked lists examples 2/2

drivers/i2c/busses/i2c-stm32f7.c

```
static int stm32f7_i2c_compute_timing(/* ... */)
{
  struct stm32f7_i2c_timings *v;
  struct list_head solutions;
  INIT_LIST_HEAD(&solutions);
  /* ... */

  for (p = 0; p < STM32F7_PRESC_MAX; p++) {
    for (l = 0; l < STM32F7_SCLDEL_MAX; l++) {
      v = kmalloc(sizeof(*v), GFP_KERNEL);
      v->presc = p;
      v->scldel = l;
      list_add_tail(&v->node, &solutions);
    }
  }

  list_for_each_entry(v, &solutions, node) {
    /* ... */
  }
}
```